# Benchmarking de Inferência do YOLO Quantizado em MCU e SBC

Desempenho e eficiência energética do YOLOX-Nano int8 para detecção de EPIs em *Edge AI*

Projeto de Pesquisa — Trilha PTT6 (protocolo experimental v1.4)

Filipe Matias
Guilherme Lucas

Orientador: Prof. Herbert Oliveira Rocha

Mestrado Profissional em Computação Aplicada (PPCA) — Universidade Federal de Roraima

Disciplina: Arquitetura e Organização de Computadores

Outubro de 2026

**Resumo**

Este projeto compara, de forma reprodutível, a execução de um mesmo modelo de detecção de objetos — YOLOX-Nano quantizado em int8 — em dois extremos do espectro *Edge AI*: um microcontrolador ESP32 (Xtensa LX6) e um computador de placa única Raspberry Pi Zero 2 W. A tarefa é a detecção de Equipamentos de Proteção Individual (EPIs) no conjunto Ultralytics Construction-PPE, começando pela classe colete e ampliando para as 11 classes. Medem-se latência ponta a ponta (pré, inferência e pós-processamento), throughput, energia por inferência — estimada por software a partir de um modelo de potência por modo e dos tempos medidos por fase —, inferências por joule, EDP e mAP. A hipótese H1 afirma que o SBC tem throughput maior, mas energia por inferência igual ou superior. O desenho prevê piloto para definir repetições e warm-up, coleta em blocos independentes, estatística robusta (Mann-Whitney U, $\delta$ de Cliff e IC por *bootstrap*) e análise de sensibilidade do modelo de potência.

**Palavras-chave:** Edge AI; TinyML; quantização int8; YOLOX; benchmarking; eficiência energética.

## 1 Introdução

A inferência de redes neurais na borda (*Edge AI*) desloca o processamento da nuvem para dispositivos próximos ao sensor, reduzindo latência, tráfego e exposição de dados. No outro lado, impõe restrições severas de memória, capacidade de computação e energia. Entre as plataformas de borda, dois extremos se destacam: **microcontroladores (MCU)**, de consumo muito baixo e memória da ordem de centenas de kilobytes, e **computadores de placa única (SBC)**, que executam um sistema operacional completo e oferecem mais desempenho a um custo energético maior.

O monitoramento do uso de EPIs em canteiros de obra é uma aplicação em que essa escolha é concreta: a câmera precisa operar continuamente, frequentemente com alimentação restrita. Falta, porém, uma comparação direta e reprodutível do *mesmo* detector quantizado nas duas classes de plataforma.

### 1.1 Pergunta de pesquisa e hipóteses

**Pergunta:** como se comparam latência, throughput e energia por inferência entre uma MCU (ESP32 LX6) e um SBC (Raspberry Pi Zero 2 W) ao executar o mesmo YOLOX-Nano int8 na detecção de EPIs?

**H1:** o SBC apresenta throughput maior, mas energia por inferência igual ou superior, dado o seu consumo estático mais alto.

**H0:** não há diferença nas métricas entre as plataformas.

### 1.2 Objetivos

**Geral:** comparar desempenho e eficiência energética da inferência do YOLOX-Nano int8 em uma MCU e em um SBC. **Específicos:**

1. preparar um artefato único `.tflite` int8 e validar sua equivalência funcional nas duas plataformas;
2. medir latência por fase, throughput e acurácia (mAP) sob um protocolo declarado;
3. estimar a energia por inferência por um modelo de potência parametrizado e quantificar sua sensibilidade;
4. testar H1 com estatística robusta e publicar os dados brutos (CSV) e o código.

### 1.3 Vínculo com a disciplina

A comparação mobiliza desempenho e a Lei de Amdahl (tempos entre arquiteturas e o ganho de paralelismo nos quatro núcleos do SBC), a hierarquia de memória (SRAM/PSRAM do ESP32 versus DRAM do SBC) e E/S.

## 2 Fundamentação e trabalhos relacionados

Benchmarks de inferência embarcada como o MLPerf Tiny [1] padronizam tarefas, métricas (latência e energia) e regras de medição para microcontroladores, medindo apenas a inferência. Noronha et al. [2] propõem integrar energia e latência separando três fases (pré-inferência, inferência e pós-inferência) e adotam o EDP como métrica de eficiência — abordagem que este projeto segue na janela de medição. Fixa-se o runtime (TFLite int8) e varia-se apenas a plataforma, para isolar o efeito do hardware. Para a metodologia de repetições, seguimos a recomendação de definir o número de execuções a partir da variabilidade observada, e não de um valor fixo [3]. O YOLOX [4] é um detector *anchor-free*; sua variante Nano tem $\approx$0,91 M parâmetros e 1,08 GFLOPs a $416 \times 416$.

# 3 Materiais

## 3.1 Plataformas

Tabela 1: Plataformas comparadas (fator independente).

| | **MCU** | **SBC** |
|---|---|---|
| Dispositivo | ESP32 (Xtensa LX6, dual-core) | Raspberry Pi Zero 2 W (Cortex-A53, 4 núcleos) |
| Clock | 240 MHz, sem DFS/light-sleep | 1 GHz, governor `performance` |
| Memória | 520 KB SRAM + PSRAM externa [confirmar módulo] | 512 MB LPDDR2 |
| Runtime | TensorFlow Lite Micro (int8) | TFLite/LiteRT (int8), 4 threads |
| Alimentação | USB 5 V | fonte oficial 5 V/2,5 A |

A proposta original citava o ESP32-S3; adotou-se o ESP32 (LX6) disponível (emenda v1.1), sem as extensões vetoriais do S3. Os resultados não se extrapolam para o S3.

## 3.2 Modelo

YOLOX-Nano na variante *ti-lite* (amigável à quantização), treinado uma única vez por configuração de classes no conjunto de treino (fora do escopo medido) e quantizado pós-treino em int8 *full-integer* com 750 imagens de calibração estratificadas. O mesmo arquivo `.tflite`, com hash SHA-256 registrado, é executado nas duas plataformas.

## 3.3 Conjunto de dados

Ultralytics Construction-PPE [5] (AGPL-3.0), com 11 classes: *helmet*, *gloves*, *vest*, *boots*, *goggles*, *none*, *Person*, *no_helmet*, *no_goggle*, *no_gloves* e *no_boots*. O bloco A usa só a classe *vest*; o bloco B, as 11 classes. A Tabela 2 resume as contagens observadas no arquivo baixado.

Tabela 2: Construction-PPE: imagens e instâncias da classe *vest* por split.

| Split | Imagens | Instâncias *vest* | Imagens com *vest* |
|---|---|---|---|
| Treino | 1142 | 1283 | 837 |
| Validação | 143 | 171 | 109 |
| Teste | 141 | 178 | 111 |

# 4 Metodologia experimental

## 4.1 Estratégia em fases

O trabalho avança em fases com critério de saída: **F1** validação do modelo com imagens estáticas (sem câmera); **F3** coleta das medições; **F4** análise e entrega. A integração da câmera ao vivo (**F2**) passou a ser um *extra* opcional (emenda v1.4): não bloqueia a medição nem entra no teste de H1. Nenhuma medição final ocorre antes de a F1 estar funcional. Estado atual: o modelo já executa no Raspberry Pi; a integração no ESP32 e a instrumentação final estão em andamento.

## 4.2 Variáveis

Tabela 3: Variáveis do experimento.

| Tipo | Variáveis |
|---|---|
| Independente | Plataforma $\in$ {MCU, SBC} |
| Dependentes | $t_{pre}$, $t_{inf}$, $t_{post}$, latência $t_{total}$ (ms); throughput (inf/s); energia por inferência $E$ (mJ); inferências/J; EDP (mJ·ms); mAP@50 e mAP@50-95 |
| Controladas | artefato (hash), resolução comum $r$, quantização, limiares (conf. $\geq$0,25; IoU$_{NMS}$ 0,45; até 100 detecções), mesmas 50 entradas, clock/governor, threads, rádio, alimentação, ambiente |
| Covariáveis | temperatura da CPU, frequência e *throttling* (SBC), temperatura ambiente, nº de detecções, imagem, bloco e ordem de execução |

## 4.3 Pipeline e janela de medição

A latência cobre a janela completa *pré + inferência + pós*, e as três fases são registradas separadamente. Na fase estática, as entradas são tensores brutos $r \times r \times 3$ uint8 já em *letterbox*, idênticos byte a byte nas duas plataformas; o pré-processamento medido é a cópia e a quantização para o tensor de entrada. O pós-processamento é a decodificação da saída do YOLOX e o NMS por classe, com a mesma lógica nas duas pontas. A resolução $r$ é comum: a maior de $\{416, 320, 256, 224, 160\}$ que caiba na memória do ESP32 e permita ao menos 30 inferências por bloco em até 60 min.

## 4.4 Ingestão direta de imagens

Validação e medição usam as mesmas imagens do conjunto de teste, pré-processadas uma vez no PC e *ingeridas diretamente no modelo*. No ESP32 (core Arduino ou ESP-IDF com TensorFlow Lite Micro), o PC envia cada imagem pela serial USB-UART em um quadro com cabeçalho e CRC32; o firmware a guarda em um buffer em RAM, confirma o recebimento e só então abre a janela cronometrada; os tempos e as detecções voltam pela serial depois da janela. No Raspberry Pi, as imagens são lidas do disco para a RAM antes do laço. Assim, a janela começa com a imagem já em memória nas duas plataformas, e o custo de transferência fica fora da medição. A serial foi preferida à flash porque 50 imagens a $160 \times 160$ ocupam $\approx$3,8 MB, mais que a partição de dados típica do ESP32.

## 4.5 Instrumentação

Os tempos vêm de relógios monotônicos de alta resolução em software (`time.perf_counter_ns()` no SBC; `esp_timer_get_time()` no MCU). Não há E/S dentro da janela: as medidas ficam em memória e são gravadas entre blocos. Telemetria (temperatura, frequência, *throttling*) é lida fora da janela. Cada execução gera um CSV com uma linha por inferência e um arquivo de metadados (versões, configurações, hashes, ambiente).

## 4.6 Experimentos

Tabela 4: Experimentos (repetidos para os blocos A — 1 classe — e B — 11 classes).

| Id | Fase | Descrição e saída |
|---|---|---|
| E0 | F1 | Acurácia offline fp32 × int8 no teste (141 imagens), por $r$: mAP@50, mAP@50-95 |
| E1 | F1 | Equivalência funcional: mesmas 50 entradas em PC, SBC e MCU; mesmas classes e nº de detecções, IoU ≥0,95, $\lvert\Delta\text{score}\rvert \leq 1$ passo de quantização |
| E2 | F1/F3 | Piloto: viabilidade de $r$ no MCU; 100 inferências por plataforma; warm-up $W$ e $N$ |
| E3 | F3 | Benchmark principal estático: 3 blocos por plataforma, $N = \max(30, N_{piloto})$ |
| E3b | F3 | SBC com 1, 2 e 4 threads: fração paralelizável por Amdahl (exploratório) |
| E3c | F3 | SBC a $416 \times 416$ quando $r < 416$ (exploratório) |
| E4 | F3 | Throughput sustentado em 60 s de laço fechado e deriva térmica |
| E5 | extra | Câmera ao vivo (opcional, fora de H1): FPS sustentado e latência com captura |
| E6 | F4 | Energia derivada, sensibilidade e *break-even* |

**Warm-up e número de repetições.** Em janelas de 10 inferências, descartam-se as iniciais até a variação relativa da mediana entre janelas consecutivas ficar abaixo de 2% em duas janelas seguidas (piso de 5). Com o coeficiente de variação (CV) de $t_{total}$ medido no piloto,

$$N = \left\lceil \left( \frac{1{,}96\,\mathrm{CV}}{\varepsilon} \right)^2 \right\rceil, \qquad \varepsilon = 1\%, \qquad N_{final} = \max(30, N), \tag{1}$$

podendo $N$ diferir entre plataformas.

**Estrutura de um bloco (E3).** Religar o dispositivo e aguardar 2 min ocioso (registrar temperatura e *throttling*); executar $W$ inferências de warm-up (marcadas e excluídas); executar $\lceil N_{final}/3 \rceil$ inferências cronometradas, percorrendo as 50 entradas em ordem fixa; gravar CSV e metadados; aguardar o resfriamento (temperatura de volta a ±2 °C da inicial no SBC, ou 5 min no MCU). Os blocos são intercalados entre plataformas para diluir efeitos de dia e ambiente.

## 4.7 Modelo de energia

Como a medição física (INA219) foi descartada por aumentar demais a complexidade e o prazo, a energia é estimada por software:

$$E_i = P_{pre}\, t_{pre,i} + P_{inf}\, t_{inf,i} + P_{post}\, t_{post,i}, \tag{2}$$

com a potência total do dispositivo em cada modo (Tabela 5).

Tabela 5: Parâmetros do modelo de potência (nominal e faixa de sensibilidade).

| Plataforma | Modo | Nominal | Faixa | Fonte |
|---|---|---|---|---|
| SBC (placa, 5 V) | ocioso | 0,60 W | 0,50–1,00 W | [6, 7] |
| SBC | ativo, 1 núcleo | 1,00 W | 0,75–1,30 W | [7] |
| SBC | ativo, 4 núcleos | 2,30 W | 1,85–2,90 W | [7, 8] |
| MCU (chip, 3,3 V) | ativo, 240 MHz | 0,224 W | 0,099–0,224 W | [9] |
| MCU | *overhead* da placa | 0 W | 0–0,25 W | parâmetro declarado |

Com potência constante por modo, $E$ é uma transformação linear dos tempos; a incerteza do modelo é tratada por sensibilidade, não pela estatística. A condição de *break-even* é

$$E_{SBC} \geq E_{MCU} \iff \frac{P_{SBC}}{P_{MCU}} \geq \frac{t_{MCU}}{t_{SBC}}, \tag{3}$$

e reporta-se se a razão de potências necessária cai dentro da faixa plausível. Como os valores do SBC são de placa e os do MCU de chip, H1 é avaliada também no pior caso para o MCU. Para um cenário de aplicação a $f$ quadros/s, reporta-se ainda $E_{ciclo}(f) = P_{ativo}\, t + P_{ocioso}(1/f - t)$, válido para $t < 1/f$.

### 4.8 Análise estatística

Por plataforma, reportam-se mediana e IQR, média ± desvio-padrão e CV das inferências válidas de E3. A comparação usa o teste de Mann-Whitney U (bilateral para latência e unilateral nas direções de H1), o $\delta$ de Cliff e intervalos de confiança de 95% por *bootstrap* BCa (10 000 reamostras) para a diferença e a razão das medianas. H1 é avaliada como teste de interseção–união, com $\text{throughput}_{SBC} > \text{throughput}_{MCU}$ e $E_{SBC} \geq E_{MCU}$, cada parte a $\alpha = 0{,}05$; a parte energética é avaliada no valor nominal e nos extremos da Tabela 5. Outliers não são removidos; a remoção por Tukey (1,5×IQR) entra apenas como sensibilidade. Verificam-se o efeito de bloco (Kruskal-Wallis) e a autocorrelação (ACF de *lag* 1). Uma análise secundária usa apenas $t_{inf}$, isolando o núcleo de inferência das diferenças de implementação do pré e do pós-processamento.

## 5 Ameaças à validade

Tabela 6: Principais ameaças e mitigação.

| **Ameaça** | **Mitigação** |
|---|---|
| Energia estimada, não medida | mesmo método nas duas plataformas; premissas registradas; sensibilidade e *break-even* |
| Escopos distintos (placa × chip) | *overhead* de placa do MCU como parâmetro; H1 no pior caso para o MCU |
| Throttling térmico no SBC | governor fixo, dissipador, pausas; bloco com *throttling* é repetido |
| Pré/pós em linguagens diferentes | mesma lógica e limiares; análise secundária só com $t_{inf}$ |
| ESP32 LX6 em vez de S3 | desvio declarado; sem extrapolação para o S3 |
| Resolução reduzida pela MCU | mesma $r$ nas duas plataformas; SBC a 416 apenas exploratório |
| Ingestão por serial (MCU) × disco (SBC) | transferência fora da janela; janela inicia com a imagem em RAM |
| Câmera ao vivo (extra) | fora do teste de H1; se feita, mesma resolução, formato e nº de buffers |
| Autocorrelação e deriva | blocos independentes, ordem registrada, ACF e efeito de bloco |

## 6 Cronograma e estado atual

Tabela 7: Cronograma (15 semanas) e estado em outubro de 2026.

| Semanas | Fase | Estado |
|---|---|---|
| 1–2 | Revisão e protocolo | concluída (protocolo v1.4; hardware e dataset definidos) |
| 3–5 | Implementação | SBC executando o modelo; ESP32 e instrumentação em andamento |
| 6–9 | Experimentos | E2 (piloto) assim que $r$ e os artefatos forem fixados |
| 10–11 | Análise | — |
| 12–13 | Relatório | — |
| 14–15 | Vídeo e entrega | — |

## 7 Entregáveis

Repositório versionado com licença MIT; README de reprodutibilidade; dados brutos em CSV e metadados; vídeo de demonstração (3–5 min) nas duas plataformas, mostrando a ingestão de imagens e as detecções; relatório técnico conclusivo. Após a disciplina: dataset com DOI (Zenodo) e submissão a evento regional.

## Referências


[1] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, et al. MLPerf Tiny benchmark. *arXiv preprint arXiv:2106.07597*, 2021.

[2] Noronha et al. Benchmarking energy and latency in TinyML. *arXiv preprint arXiv:2505.15622*, 2025. Completar a lista de autores antes da versão final.

[3] Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. Taming performance variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 409–425, 2018.

[4] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.

[5] Ultralytics. Construction-PPE detection dataset. `https://docs.ultralytics.com/datasets/detect/construction-ppe/`, 2025. Licença AGPL-3.0. Acesso em: out. 2026.

[6] CNX Software. A deep dive into Raspberry Pi Zero 2 W's power consumption. `https://www.cnx-software.com/2021/12/09/raspberry-pi-zero-2-w-power-consumption/`, 2021.

[7] Jeff Geerling. Disabling cores to reduce the Pi Zero 2 W's power consumption by half. `https://www.jeffgeerling.com/blog/2021/disabling-cores-reduce-pi-zero-2-ws-power-consumption-half/`, 2021.

[8] Tom's Hardware. Raspberry Pi Zero 2 W review: The long awaited sequel. `https://www.tomshardware.com/reviews/raspberry-pi-zero-2-w-review`, 2021.

[9] Espressif Systems. *ESP32 Series Datasheet, versão 5.3*. Tabela 4-2, Power Consumption by Power Modes.